Smart Agriculture Advisory System

Weekly Development Report

Abhijit Dalal | ESS | Roll No. 241060901

1. Executive Summary

This report covers the development and training of four machine learning models that form the core of the Smart Agriculture Advisory System — a Raspberry Pi-based precision farming assistant. The system integrates real-time soil and weather sensor data with trained XGBoost models to deliver actionable crop, yield, irrigation, and fertilizer recommendations. Three of the four models are production-ready.

2. System Architecture & Hardware

All models run inference on a Raspberry Pi loaded with a Flask REST API. Sensor readings are passed directly into the prediction pipeline, and the OpenClaw actuator responds to irrigation model outputs automatically.

Hardware

- Raspberry Pi — primary compute unit for inference
- DHT22 sensor — temperature and humidity readings
- Soil moisture sensor — real-time moisture percentage
- Soil pH sensor — pH level monitoring
- OpenClaw — configured and operational for automated irrigation actuation

Software

- Python 3.13 | XGBoost, scikit-learn, pandas, numpy
- Flask REST API serving all four model endpoints
- Model persistence: .json (XGBoost native) + .pkl (encoders / metadata)

3. Datasets

Module	Kaggle Dataset	Rows	Key Features
Crop Recommendation	atharvaingle/crop-recommendation-dataset	2,200	N, P, K, temp, humidity, pH, rainfall
Yield Prediction	akshatgupta7 + zoya77 (merged)	26,000+	State, crop, season, area
Smart Irrigation	miadul/irrigation-requirement-prediction	10,000	Soil type, pH, moisture, weather
Fertilizer Rec.	irakozekelly/fertilizer-prediction	100,000	NPK, soil type, crop type, moisture

Note: For crop recommendation, if the farmer does not know their NPK values, soil color can be provided. A soil color-to-NPK approximation mapping is applied to generate the required input features.

4. Model 1 — Crop Recommendation

Recommends the optimal crop from 22 classes based on soil NPK values, temperature, humidity, pH, and rainfall. XGBoost multi-class classification with stratified 80/20 train-test split.

Metric	Value	Status
Test Accuracy	**98.86%**	Excellent
Train / Test Split	**1,760 / 440**	80/20 Stratified
Classes	**22 crops**	All Indian staples
Missing Values	**0**	Clean dataset

```
model = xgb.XGBClassifier(
    n_estimators=300, max_depth=6, learning_rate=0.1,
    subsample=0.8, colsample_bytree=0.8,
    eval_metric='mlogloss', random_state=42, n_jobs=-1
)

def predict_crop(N, P, K, temperature, humidity, ph, rainfall):
    df = pd.DataFrame([[N, P, K, temperature, humidity, ph, rainfall]],
        columns=['N','P','K','temperature','humidity','ph','rainfall'])
    return le.inverse_transform(crop_model.predict(df))[0]
```

5. Model 2 — Crop Yield Prediction

Predicts crop yield (tonnes/hectare) for Indian states using XGBoost regression. Merges two Kaggle sources for broader geographic and seasonal coverage. Categorical features are one-hot encoded.

Metric	Value	Status
R^2 Score	**0.9385**	Excellent
RMSE	**221.9954**	Acceptable range
MAE	**15.9269**	Low error
Config	**500 trees, depth=8, lr=0.05**	XGBRegressor

```
yield_model = xgb.XGBRegressor(
    n_estimators=500, max_depth=8, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    objective='reg:squarederror', eval_metric='rmse', random_state=42
)

def predict_yield(row):
    row = pd.get_dummies(row.drop(columns=artifacts['dropped_columns']))
    row = row.reindex(columns=artifacts['feature_columns'], fill_value=0)
```

```
    return loaded_yield_model.predict(row)[0]
```

6. Model 3 — Smart Irrigation

Classifies irrigation need into Low / Medium / High based on soil and weather conditions. Integrated with the OpenClaw actuator on Raspberry Pi for automated irrigation response.

Metric	Value	Status
Test Accuracy	**73.70%**	Deployed — improvable
Classes	**Low / Medium / High**	3-class
Train / Test Split	**8,000 / 2,000**	80/20 Stratified
OpenClaw	**Configured**	Operational

Improvement note: Accuracy can be increased by re-adding Crop_Type, Sunlight_Hours, and Field_Area as features in the next training run. These were dropped for the initial run.

```
irrigation_model = xgb.XGBClassifier(
    objective='multi:softprob', num_class=3,
    n_estimators=200, max_depth=6,
    subsample=0.8, colsample_bytree=0.8, random_state=42
)

# OpenClaw triggered when irrigation_model returns 'High'
def trigger_openclaw(level):
    if level == 'High':
        pass  # GPIO / serial command to actuate OpenClaw
```

7. Model 4 — Fertilizer Recommendation

Recommends one of 7 fertilizers (10-26-26, 14-35-14, 17-17-17, 20-20, 28-28, DAP, Urea) based on soil, crop, and environmental inputs. Trained on 100,000 samples.

Metric	Value	Status
Test Accuracy	**14.38%**	Near-random — needs fix
Training Samples	**80,000**	Large dataset
Val. Loss Trend	**Increasing (1.946 → 1.979)**	Model not learning
Random Baseline	**14.3% (1/7)**	No improvement seen

Issue: Validation mlogloss increased throughout training — the model learned nothing useful. All 7 fertilizer classes show identical precision/recall (~0.14). The dataset appears to have near-uniform label distribution with no strong predictive signal in the features.

Fix plan: Replace dataset with one that has stronger crop-soil-NPK-fertilizer correlations, or add target encoding for the Crop Type x Soil Type interaction.

```
fertilizer_sample = {
    'Temparature': 33, 'Humidity': 56, 'Moisture': 56,
    'Soil Type': 'Loamy', 'Crop Type': 'Ground Nuts',
    'Nitrogen': 37, 'Potassium': 5, 'Phosphorous': 24
}
predict_fertilizer(fertilizer_sample)  # => '28-28'
```

8. Raspberry Pi & Flask Integration

All four models are saved to ./model/ and loaded via Flask on startup. The OpenClaw actuator is configured and responds to irrigation model output.

```
./model/
  crop_recommendation.json / .pkl
  crop_yield.json / .pkl
  irrigation_model.json / .pkl
  fertilizer_recommendation.json / .pkl
```

```
# app.py — Flask API on Raspberry Pi
import xgboost as xgb, pickle, pandas as pd
from flask import Flask, request, jsonify

app = Flask(__name__)
crop_model = xgb.XGBClassifier()
crop_model.load_model('./model/crop_recommendation.json')

@app.route('/recommend/crop', methods=['POST'])
def recommend_crop():
    data = request.json
    df = pd.DataFrame([data])
    pred = crop_model.predict(df)[0]
    label = crop_artifacts['label_encoder'].inverse_transform([pred])[0]
    return jsonify({'recommended_crop': label})
```